

Lecture Unit 5.1

Floating point arithmetic

Integers

- Integers can be represented exactly
- For example an unsigned 8-bit (1 byte) integer

2^7 =128	2^6 =64	2^5 =32	2^4 =16	2^3 =8	2^2 =4	2^1 =2	2^0 =1
1	0	0	1	0	1	0	1

$$2^7 + 2^4 + 2^2 + 2^0 = 128 + 16 + 4 + 1 = 149$$

Integers

- Integers are represented exactly
- There is a finite range of integers which can be represented
 - Maximum unsigned integer with n bits: $2^n - 1$
- Negative integers can be represented (e.g. using two's complement)

Why floating point?

- In week 3 we saw how a continuous PDE can be solved using a finite difference approximation
- The solution is a continuous variable evaluated at discrete points
- The solution itself is not restricted to integer values so we need a way to represent non-integer values using bits

Why floating point?

- We want to be able to handle very large and very small numbers:
 - Mass of the electron: $9.109 \times 10^{-31} \text{ kg}$
 - Mass of the Sun: $1.99 \times 10^{30} \text{ kg}$
- More than 60 orders of magnitude between these numbers
- We need a representation where the decimal point can move (floating point)

Floating point representation

- The floating point representation has a base (β) and a precision (p)
- A floating point number is represented by $d . dddd \times \beta^e$:
 - $d . dddd$ is the significand (mantissa) which has p digits
 - e is the exponent
- For example $9.109 \times 10^{-31} \text{ kg}$

$$d . ddd = 9.109$$

$$e = -31$$

$$\beta = 10$$

$$p=4$$

IEEE 754

- Floating point is the most widely used method for representing non-integer values
- IEEE 754 is the most widely adopted standard for floating point arithmetic
- The standard specifies number representations (e.g. single precision, double precision)
- The standard also specifies how addition, subtraction, multiplication, division and square root operations should behave

IEEE 754 support in C

- Support for IEEE 754 was added to C in the C99 standard
- The two most commonly used precisions are single precision and double precision:
 - Single precision: 32-bit (`float`)
 - Double precision: 64-bit (`double`)

Why double precision?

- Floating point numbers are an approximation to the real numbers
- Floating point numbers have a finite accuracy and a finite range
- We need to consider the range and accuracy required for a given application
- Double precision floating point arithmetic is the standard in scientific and engineering calculations
- Other applications may not require as much precision (e.g. machine learning, graphics)

IEEE 754 double precision

- Double precision uses 64-bits (8-bytes)
- There are 52 bits for the mantissa, 11 bits for exponent and 1 bit for the sign ($\pm$)
- If the exponent bits are neither all zeros or all ones then the number is said to be a “normalised” floating point number
- A normalised floating point number x is represented by:

$$x = \pm (1.b_1 b_2 \dots b_{52})_2 \times 2^{(a_1 a_2 \dots a_{11})_2 - 1023}$$


where $b_1 b_2 \dots b_{52}$ are the mantissa bits, $a_1 a_2 \dots a_{11}$ are the exponent bits, and $(\dots)_2$ means a binary number

Range

- $x = \pm (1.b_1 b_2 \dots b_{52})_2 \times 2^{(a_1 a_2 \dots a_{11})_2 - 1023}$

- Smallest normalised number: all mantissa bits are 0, all exponent bits are 0 except a_{11}

$$1 \times 2^{1-1023} = 2^{-1022} \approx 10^{-308}$$

- Largest normalised number: all mantissa bits are 1, all exponent bits are 1 except a_{11}

$$(2 - 2^{-52}) \times 2^{2046-1023} \approx 10^{308}$$

Accuracy

- $x = \pm (1.b_1 b_2 \dots b_{52})_2 \times 2^{(a_1 a_2 \dots a_{11})_2 - 1023}$
- The difference between 1 and the next largest representable floating point number is often called the “machine epsilon”
- The place value of b_{52} is 2^{-52} so the machine epsilon is $2^{-52} \approx 10^{-16}$ for double precision
- These rounding errors will accumulate so the overall effect depends on the number of floating point operations in our algorithm

Round-off error

- In week 3 we saw that finite difference approximations have a truncation error
- Other numerical methods (e.g. calculating a series solution) will have associated numerical errors
- There is also a round-off error from the floating point representation of the numbers
- If the error from the numerical method is smaller than the error from the floating point representation then the solution is said to be calculated to “machine precision”

Unit Summary

- Floating point is used to represent non-integer values
- IEEE 754 is the most common standard for floating point arithmetic
- IEEE 754 double precision uses 64 bits (8 bytes) to represent the number
- Most scientific calculations use double precision arithmetic (**double** in the C programming language)
- The range of normalised double precision numbers is 10^{-308} to 10^{308}
- The finite precision of floating point numbers introduces a round-off error into calculations